

Declarative Command Lines

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 2

Who this book is for . . . 2

A book that stands on its own . . . 2

What you need . . . 3

Conventions . . . 3

I: The Why . . . 4

1. Command lines that assemble themselves . . . 5

1.1. Contributed, not listed . . . 5

1.2. Where a command goes . . . 5

1.3. Pipelines . . . 6

1.4. For people, pipelines and agents . . . 6

2. Standing on picocli . . . 7

2.1. A domain, expressed . . . 7

2.2. What picocli does, and the runtime does not redo . . . 8

2.3. Bundled, for now . . . 8

2.4. Where picocli stops . . . 8

3. What Nasdanika adds: assembly by type . . . 9

3.1. Declarative assembly through capabilities . . . 9

3.2. From classes to types . . . 9

3.3. Matching is checked twice . . . 10

3.4. A trained eye . . . 10

II: The How . . . 11

4. Getting started . . . 12

4.1. Adding the module . . . 12

4.2. The module descriptor . . . 12

4.3. Running . . . 13

5. Commands and factories . . . 14

5.1. Commands . . . 14

5.2. Factories . . . 16

5.3. Assembling the tree . . . 17

5.4. Running . . . 18

5.5. Closing . . . 18

5.6. Limits . . . 19

6. Wiring by type . . . **20**
6.1. One class, two types . . . 20
6.2. What a command needs from its parent . . . 20
6.3. What is expensive, ask for lazily . . . 21
6.4. What a command accepts as sub-commands . . . 23
6.5. Offers and hand-downs . . . 24
6.6. When a parent cannot provide . . . 25
6.7. When wiring happens . . . 25
6.8. Rules . . . 26
7. Mix-ins . . . **27**
7.1. A contributed mix-in . . . 27
7.2. Mix-ins by type . . . 28
7.3. Matching mix-ins . . . 30
7.4. Mix-ins with fields . . . 30
7.5. Mix-in or command . . . 30
8. Overriding . . . **32**
8.1. By subclassing . . . 32
8.2. By declaration . . . 32
8.3. The rules . . . 33
9. The runtime's commands . . . **34**
9.1. Commands . . . 34
9.2. The shell . . . 34
9.3. Mix-ins . . . 35
9.4. Telemetry . . . 35
9.5. Distributions and the launcher . . . 35
III: Looking Ahead . . . **37**
10. Pipelines . . . **38**
10.1. What the previous generation built . . . 38
10.2. Being rebuilt on types . . . 38
10.3. Planned . . . 38
Appendix A: Reference . . . **40**
A.1. Writing commands . . . 40
A.2. Contributing . . . 40
A.3. Running . . . 41
A.4. System properties . . . 41
Appendix B: Building this book . . . **42**

Appendix C: Glossary **43**

Colophon

Declarative Command Lines: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the command line module of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime:cli:2026.11.0`, Java module `org.nasdanika.sdk.runtime.cli`. Every listing in it is included from the module's sources and tests, which are compiled and run when the book is built.

Copyright © 2026 Pavel Vlasov.

The text of this book is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt it for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Declarative Command Lines: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution. Picocli, which the runtime bundles, is licensed under the Apache License 2.0.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

Java is a trademark of Oracle and its affiliates. Eclipse and EMF are trademarks of the Eclipse Foundation. OpenTelemetry is a trademark of the Linux Foundation. Spring is a trademark of Broadcom. Other names may be trademarks of their owners. Their use here identifies the technologies discussed and implies no endorsement.

The source of this book, its tests and the module it describes are at github.com/Nasdanika-SDK/runtime, in `cli`.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about command lines that assemble themselves. It describes `cli`, the module of the Nasdanika SDK Runtime that builds a picocli command tree from whatever modules are on the module path: each module contributes commands and mix-ins, and each command says, with generic types, what it needs from its parent and what it accepts as sub-commands. Nothing lists all the commands; the tree is what the modules contribute, and a command appears exactly where its declared needs are met.

The first part is the why: why a platform's command line should assemble itself, what picocli does so well that the runtime builds on it rather than beside it, and what Nasdanika adds. The second part is the how. The third is about the pipelines the approach makes possible.

Who this book is for

Java developers who build command line tools that grow by modules: a core, and features, formats and integrations that come and go with the dependencies.

Developers of the runtime's modules and of modules built on it, who contribute commands to `nsdk`, the runtime's command line, or to a command line of their own.

Architects who want a command line as the first user interface of a platform, for people, for build pipelines and for AI agents, with every run recorded as telemetry. Part one is for you, and needs no code.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth manuals, one per module of the Nasdanika SDK Runtime, in the manner of IBM Redbooks. The runtime's own book, *Nasdanika SDK Runtime: More Structure with Less Effort*, introduces the command line in three chapters, and the runtime's `common` module, on which the wiring is built, has a Zephyr Book of its own, *Matching and Wiring*. You do not need either to read this one: where the books touch, this one repeats what you need.

The series is named after the Bootswatch theme it is styled with, Zephyr, and has nothing to do with the Zephyr real-time operating system or with SmartBear's Zephyr test management.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later**.
- The module is `org.nasdanika.sdk.runtime.cli`, Maven coordinates `org.nasdanika.sdk.runtime:cli:2026.11.0`.
- Some familiarity with picocli helps: its annotations are how commands are written. Its documentation, picocli.info, is excellent.

Conventions

- **Every listing is included from the module's sources or tests** and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Status labels** mark what this edition does not have yet: PLANNED for designed but not built, SKETCH for partly built.
- **Voice.** I write in the first person.



> Editor's notes record inconsistencies between the module and its documentation found while writing. They are shown in builds for the author and left out of published editions.

Part I: The Why

Why a command line should assemble itself from the modules it runs with, what picocli already does superbly, and what Nasdanika adds on top of it. No code is needed to read this part.

Chapter 1. Command lines that assemble themselves

A command line tool usually knows all of its commands. The main class lists them, or an annotation does, and adding a command means changing the tool. That is right for a tool with one purpose and one team.

A platform is different. Its capabilities come from modules: a model, a format, a diagram editor’s documents, a cloud provider’s API, a site generator, each in its own module, written by different people, released at different times. Its command line should be the sum of the modules it runs with, and a distribution for one audience should have the commands of its modules and no others. The command line of such a platform cannot be listed in one place, because no one place knows all of it.

1.1. Contributed, not listed

In the Nasdanika SDK Runtime a module contributes commands the way it contributes anything else: by declaring a provider of a service in its module descriptor. The runtime collects the contributions with its capability framework and builds the command tree from them. Add a module to a distribution and its commands appear; remove it and they are gone, with no other change.

1.2. Where a command goes

Contribution answers which commands exist. The harder question is where each one goes.

A command that saves a model to a file needs a model to save. It does not care where the model comes from: a file in any format, a Draw.io diagram converted to a model, a model generated from a database, the result of a filter. In a fixed tool, the save command is written as a sub-command of each of those, by whoever wrote them. In a contributed command line, the save command says what it needs, "a parent that can give me model objects", and goes under every command that can.

That is the idea of this module. A command declares what it needs from its parent and what it accepts as sub-commands; the runtime places it under every parent that provides what it needs,

before anything runs; and when it runs, the runtime hands it what its parent provides.

1.3. Pipelines

Commands placed by what they need form pipelines. In the previous generation of Nasdanika, `nsd drawio diagram.drawio html-app site` read as a sentence: load a Draw.io document, turn it into an application model, generate a site from it. Each word was a command from a different module, and none of them knew the others; each knew only the kind of thing it consumed and the kind of thing it offered to whatever followed.

New sentences come from new words. A module contributing a command that consumes application models, a PDF generator for example, would add `drawio diagram.drawio html-app pdf` without touching the Draw.io, application model or site modules. The vocabulary of the command line grows by federation, as modules are added, and the grammar, which word may follow which, is checked by the types they declare.

1.4. For people, pipelines and agents

The command line is the runtime's first user interface. People run it in a terminal; build pipelines run it in scripts; and AI agents, which can run a command and read a file, use everything it offers with no protocol to implement. For all three, a command line that lists exactly the commands that will work where they are listed, and records every run as telemetry, is the difference between a tool and a platform.

Chapter 2. Standing on picocli

The commands of this module are picocli commands, and the module would not exist without picocli.

2.1. A domain, expressed

Picocli, created by Remko Popma, is a framework for command line applications in Java. A command is a class; `@Command` names it and describes it; `@Option` and `@Parameters` on fields declare its options and parameters, typed, with defaults, arity, validation and descriptions; `@Mixin` reuses a set of options; `@ParentCommand` and `@Spec` give a command its context. From those declarations picocli parses arguments, converts them to the fields' types, generates usage help in color, version information, shell completion scripts and documentation, supports argument groups, and compiles to native images.

All of it is plain Java, without a line of parsing code in the command. When I first read picocli commands, I was super-impressed by how Remko Popma had expressed the problem domain of command lines in Java: options, parameters, sub-commands, mix-ins, help, each a Java construct that a Java developer already reads fluently. It is an internal DSL in the best sense: a language for a domain, made of the host language's own parts, so that learning the domain is learning nothing new about the language.

It is also the sensory bias at work again, the idea from evolutionary biology that a preference which evolved for one purpose is exploited for another. Java developers' eyes were trained by annotations on fields and by typed values, for persistence, for dependency injection, for web endpoints; picocli speaks to that trained eye about a different domain. Nasdanika does the same with generic types, as the next chapter explains.

The first time I heard Londoners speak, I said to myself: "Oh, this is how this language is supposed to be spoken." I had a similar feeling reading picocli: this is how a command line is supposed to be written in Java.

2.2. What picocli does, and the runtime does not redo

Everything about a single command, and about a command line once it is assembled, is picocli's:

- parsing, type conversion, defaults and validation of options and parameters;
- usage help, version help and error messages;
- mix-ins, reused sets of options;
- the command tree's model, `CommandLine` and `CommandSpec`, which the runtime builds programmatically;
- execution strategies, which decide what runs after parsing; the runtime adds one.

A command written for this module is a picocli command first, and everything in picocli's documentation applies to it.

2.3. Bundled, for now

The runtime bundles picocli in a module of its own, `org.nasdanika.sdk.runtime.picocli`, which exports the `picocli` package unchanged in its use: commands import `picocli.CommandLine` as usual. The bundle carries changes contributed by Nasdanika to picocli, and will be replaced by picocli itself when a picocli release includes them, planned for picocli 4.8.



> Name the contributions to picocli, and link to the pull request or issue, before publishing. The bundled module's descriptor and pom say only "Waiting for Picocli 4.8.0 with Nasdanika contributions".

2.4. Where picocli stops

Picocli assembles a command tree from what a command lists: `subcommands` in `@Command` names the classes of the sub-commands, or code adds them with `addSubcommand`. The parent knows its children, by class. For a tool with one team that is exactly right.

For a command line contributed by modules it is the wrong way round: the parent cannot know the children that modules written later will contribute. Picocli's programmatic API makes any other way possible, and that is where this module starts.

Chapter 3. What Nasdanika adds: assembly by type

Nasdanika adds two things on top of picocli: declarative assembly through capabilities, and matching by generic types rather than classes.

3.1. Declarative assembly through capabilities

A module contributes a command with a capability factory, provided as a service in its module descriptor. The runtime builds the tree by resolving one requirement, "the root commands", and every factory that creates a command asks, within the same resolution, for the command's sub-commands and mix-ins. The tree grows until no factory matches a new place, and arrives whole: a command line is emitted only when its sub-commands and mix-ins are attached.

The factory decides whether its command goes under a parent, and in the ordinary case it does not decide in code: it reads the command's declarations. The assembly is declarative in that sense: a command states what it needs and what it accepts, and the placement follows.

3.2. From classes to types

In the 2024 releases of Nasdanika, a command stated its parents with `@ParentCommands`, listing classes or interfaces: the save command was a sub-command of any command implementing `EObjectSupplier`. It worked, and it built the pipelines described in Chapter 10, but a class is a coarse contract. `EObjectSupplier` said "I supply model objects", and more interfaces grew around it to say what else commands supplied: `LabelSupplier` for application models, `Document.Supplier` for Draw.io documents, a `BindingsSupplier` planned for scripts. Each was a type that existed to be matched, and each new kind of thing to pass along needed one.

The runtime matches on generic types instead. A command declares a member with `@Parent`, and the member's declared type is what it needs:

- `@Parent Supplier<? extends CharSequence> text` needs a parent that supplies text;
- `@Parent Supplier<? extends EObject> model` needs a parent that supplies a model element.

`Supplier<String>` and `Supplier<EObject>` are two different types, though the same class at run time.

Erasure removes the type argument from the objects, not from the declarations: the class file keeps the generic signature of every field and method, the compiler has checked each of them, and the runtime matches them with the compiler's rules. A parent offers what it supplies through a method annotated `@Adapter`, whose generic return type is the offer: `Supplier<String> text()`. No marker interfaces, no string names, no registration: a parent that supplies text attracts the commands that consume text, and only those.

3.3. Matching is checked twice

The match is made at two times, with two kinds of evidence:

- **Before anything runs**, the tree is built from declarations: a command is placed under a parent whose class can be adapted to the type of one of its `@Parent` members, by declarations only, because no options are parsed yet. Help lists exactly the commands that will work where they are listed.
- **When a command runs**, after parsing, the parent is asked for the value, and the value is injected. A parent may still decline, and then the command fails with a message that says what it needed and what the parent provided.

This is the pattern of the compiler, applied at run time: a call is matched to a method by its signature, generics validate the match, and `@Override` validates an override. Here a command is matched to a parent by its declared needs, and the generic types validate the match.

3.4. A trained eye

Java developers read `Supplier<? extends CharSequence>` without effort: generics trained them for twenty years, on collections. The runtime asks for nothing more. A command's contract is written in the language's own type system, read with a skill every Java developer already has, and checked by the compiler where it is written. Picocli did that for options and parameters; this module does it for the shape of the command tree.

Part II: The How

Adding the module, contributing commands, wiring them by type, contributing mix-ins, overriding commands, and the commands, mix-ins and launcher the module comes with.

Chapter 4. Getting started

4.1. Adding the module

The runtime has a bill of materials, `org.nasdanika.sdk.runtime:bom`, which manages the versions of its modules and their dependencies. Import it in `dependencyManagement` with the scope `import` and the type `pom`, and depend on `org.nasdanika.sdk.runtime:cli` without a version. The module brings the runtime's `common` module, its bundled picocli, and JLine for the interactive shell.

4.2. The module descriptor

Listing 1. The module's own descriptor (`module-info.java`)

```
module org.nasdanika.sdk.runtime.cli {

    exports org.nasdanika.sdk.runtime.cli;

    requires transitive org.nasdanika.sdk.runtime.common;
    requires transitive org.nasdanika.sdk.runtime.picocli;
    requires transitive org.jline;
    requires transitive io.opentelemetry.context;
    requires io.opentelemetry.api;
    requires org.json;
    requires org.yaml.snakeyaml;

    opens org.nasdanika.sdk.runtime.cli;

    provides CapabilityFactory with
        RootCommandFactory,
        SaveModelCommandFactory,
        EcoreCommandFactory;

}
```

Three lines of it are the pattern for every module that contributes commands:

- `requires org.nasdanika.sdk.runtime.cli`, transitively if the module's own API mentions commands.
- `opens` the package of the commands. Picocli and the wiring set private fields of commands and mix-ins, options and injected parents alike, so the package must be open to them; an unqualified `opens` is the simple way, and `opens … to org.nasdanika.sdk.runtime.picocli, org.nasdanika.sdk.runtime.cli, org.nasdanika.sdk.runtime.common` the strict one.

- `provides CapabilityFactory with` the factories of the commands and mix-ins. Declaring a provider is all the registration there is.

The `cli` module provides three factories: the root command, `nsdk`, and the `save` and `ecore` commands (Chapter 9).

4.3. Running

`org.nasdanika.sdk.runtime.cli.Application` is a main class: it builds the tree from the factories of the module layer it runs in, adds the interactive shell, and executes the arguments. The runtime's distribution starts through a launcher that also records every run as telemetry; distributions are the subject of Section 9.5. In a test, the tree is built with a capability loader and run with picocli's `execute`, as the next chapter shows.

Chapter 5. Commands and factories

The examples of this chapter and the next are four commands: two that supply something, text and a model element, and two that consume one of them.

5.1. Commands

A command extends `CommandBase` or `CommandGroup` and is annotated with picocli's `@Command`.

Listing 2. A group that supplies text (`PipelineTests.java`)

```
@Command(name = "text", description = "Supplies text")
public static class TextCommand extends CommandGroup {

    @Parent
    RootCommand root; ①

    @Option(names = "--value", defaultValue = "Hello")
    String value;

    @Adapter
    public Supplier<String> text() { ②
        return () -> value;
    }

}
```

① Where the command goes: under a parent that is a `RootCommand`, the root of the tree.

② What it offers its sub-commands: a `Supplier<String>`, from the value of its option.

Listing 3. A group that supplies a model element

```
@Command(name = "model", description = "Supplies a model element")
public static class ModelCommand extends CommandGroup {

    @Parent
    RootCommand root;

    @Option(names = "--name", defaultValue = "library")
    String name;

    @Adapter
    public Supplier<EObject> model() { ①
        return () -> {
            EPackage ePackage = EcoreFactory.eINSTANCE.createEPackage();
            ePackage.setName(name);
            return ePackage;
```

```
        };
    }

}
```

① A `Supplier<EObject>`: the same class as the text group's supplier, a different type.

Listing 4. A command that prints text

```
@Command(name = "print", description = "Prints its parent's text")
public static class PrintCommand extends CommandBase {

    @Parent
    Supplier<? extends CharSequence> text; ①

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) {
        spec.commandLine().getOut().println(text.get());
        return 0;
    }

}
```

① Any parent that supplies some kind of character sequence.

Listing 5. A command that describes a model element

```
@Command(name = "describe", description = "Describes its parent's model element")
public static class DescribeCommand extends CommandBase {

    @Parent
    Supplier<? extends EObject> model; ①

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) {
        EObject eObject = model.get();
        spec.commandLine().getOut().println(eObject.eClass().getName() + " " + eObject);
        return 0;
    }

}
```

① Any parent that supplies some kind of model element.

`CommandBase` runs `execute` in a span named after the command, with the command's qualified name, class, module and module version as attributes, and hands it the span and the logger and meter of the current OpenTelemetry instance. It also closes the command's sub-commands and mix-ins that implement `Closeable` when it is closed. `CommandGroup` is a command that does nothing

itself: run alone, it fails with "Missing required subcommand", which picocli reports with the usage help.

5.2. Factories

A command is contributed by a factory:

Listing 6. A command's factory

```
public static class PrintCommandFactory extends SubCommandCapabilityFactory<PrintCommand> {

    @Override
    protected Class<PrintCommand> getCommandType() { ①
        return PrintCommand.class;
    }

    @Override
    protected Publisher<PrintCommand> doCreateCommand(
            List<CommandLine> parentPath,
            Loader loader,
            Span span,
            Logger logger,
            Meter meter) {
        return Flows.of(new PrintCommand()); ②
    }

}
```

① The command's class, whose declarations decide where it goes.

② Creating the command, once it is known to go under a parent. A publisher, so a factory can create several commands, or none, or create them later.

`SubCommandCapabilityFactory` does the rest. For each place in the tree it is asked about, it checks whether the command goes there (Chapter 6), creates it with `doCreateCommand`, wraps it in a picocli `CommandLine`, and asks, in the same resolution, for the command's own sub-commands and mix-ins.

A factory that decides in code overrides `match(parentPath)`, which sees the whole path from the root, or `createCommand`, which is called for every place and creates the command where it should be. The runtime's root command factory does that: it creates the root where the path is empty.

5.3. Assembling the tree

The tree is built by resolving one requirement:

Listing 7. Assembling a command line

```
static CommandLine assemble() {
    CapabilityLoader loader = CapabilityLoader.of( ①
        new RootCommandFactory(),
        new TextCommandFactory(),
        new ModelCommandFactory(),
        new PrintCommandFactory(),
        new DescribeCommandFactory());
    Requirement roots = Requirement.of( ②
        CommandLine.class,
        new SubCommandRequirement(List.of(), new AtomicInteger()));
    return loader.<CommandLine>loadOne(roots); ③
}
```

① A loader over the factories. In an application, `CapabilityFactorySource.serviceLoader()` gives the factories of every module on the module path.

② The requirement for root commands: a `CommandLine` with an empty parent path, and a counter shared by the whole tree.

③ A blocking load. It pumps the resolution on the calling thread until it completes, and the root it returns carries the whole tree.

Printed by walking the command lines,

Listing 8. Walking the tree

```
static String tree(CommandLine commandLine, String indent) {
    StringBuilder tree = new StringBuilder(indent)
        .append(commandLine.getCommandName())
        .append('\n');
    for (CommandLine subCommand: commandLine.getSubcommands().values()) {
        tree.append(tree(subCommand, indent + "  "));
    }
    return tree.toString();
}
```

the tree is:

Listing 9. The tree (`pipeline-tree.txt`)

```
nsdk
  launcher
```

```
  model
    describe
  text
    print
```

`launcher` is the root command's own sub-command, declared in its `@Command` (Section 9.5). `print` is under `text` and not under `model`, and `describe` the other way round, because of the type arguments of what the groups supply. A sub-command's place is computed from declarations alone, before any option is parsed.

5.4. Running

Running a command is picocli's `execute`:

Listing 10. Running commands

```
assertThat(run(assemble(), "text", "--value", "Hi there", "print")).isEqualTo("Hi there");
assertThat(run(assemble(), "model", "--name", "shelf", "describe")) ①
    .startsWith("EPackage")
    .contains("(name: shelf)");
```

① The group's option is parsed before its sub-command runs, so the supplier the sub-command gets supplies a package named "shelf".

5.5. Closing

A command that holds resources, a connection, a resource set, a temporary directory, implements `Closeable`, as `CommandBase` does: closing a command closes its sub-commands and mix-ins that implement `Closeable`, and the whole tree below them, also below commands that do not implement it.

Listing 11. A command that knows when it is closed (`LifecycleTests.java`)

```
@Command(name = "connect")
public static class ConnectCommand extends CommandGroup {

    @Parent
    RootCommand root;

    boolean closed;

    @Override
    public void close() {
```

```
        super.close(); ①
        closed = true;
    }

}
```

① Closes what is below it first.

Listing 12. Closing a command line

```
CommandLine root = ContributionTests.assemble(
    new RootCommandFactory(),
    new ConnectCommandFactory(),
    new ModelCommandFactory());
ConnectCommand connect = (ConnectCommand) root
    .getSubcommands().get("connect")
    .getCommandSpec().userObject();

assertThat(root.getCommandSpec().userObject()).isInstanceOf(RootCommand.class); ①
CommandBase.close(root); ②
assertThat(connect.closed).isTrue();
```

① The root is a plain picocli command, not `Closeable`.

② `CommandBase.close` closes a command line all the same: its command if it is `Closeable`, and otherwise its sub-commands and mix-ins. The application and the launcher call it when a run is over.

5.6. Limits

A command that takes any parent, or a pair of commands that take each other, would grow the tree forever. Two limits stop it, with a warning in the log:

- a parent path of at most 15 commands, set with the system property `org.nasdanika.sdk.runtime.cli.maxCommandPath`;
- at most 2,000 commands in a tree, set with `org.nasdanika.sdk.runtime.cli.maxCommands`.

A factory can also refuse places by itself: the runtime's `ecore` command, which filters what its parent supplies and supplies the result, refuses to go under another filter, which would otherwise nest without end.

Chapter 6. Wiring by type

6.1. One class, two types

The suppliers of the text and the model groups are the same class at run time, and different types in their declarations:

Listing 13. Types, not classes (`PipelineTests.java`)

```
TextCommand text = new TextCommand();
ModelCommand model = new ModelCommand();
Supplier<String> textSupplier = text.text();
Supplier<EObject> modelSupplier = model.model();
assertThat(textSupplier).isInstanceOf(Supplier.class); ①
assertThat(modelSupplier).isInstanceOf(Supplier.class);

Type printNeeds = new TypeToken<Supplier<? extends CharSequence>>() {}.getType();
assertThat(Adapters.isAdaptable(TextCommand.class, printNeeds)).isTrue(); ②
assertThat(Adapters.isAdaptable(ModelCommand.class, printNeeds)).isFalse();
```

① Both are `Supplier` objects, and nothing in either says what it supplies.

② The declarations do: the text group can be adapted to what `print` needs, a `Supplier<? extends CharSequence>`, and the model group cannot.

`Adapters.isAdaptable` answers from declarations: an object's own generic supertypes, and the generic types of its `@Adapter` fields and methods, matched with Java's rules for assignment. The runtime's `common` module implements it, and its Zephyr Book, *Matching and Wiring*, describes the rules in detail.

6.2. What a command needs from its parent

`@Parent` on a field, or on a method taking one parameter, says what a command needs from its parent: the member's generic type.

- **Matching**, when the tree is built: the command goes under a parent whose class is adaptable to the type of any of its `@Parent` members.
- **Injection**, after parsing and before execution: the parent is adapted to the members' types, the most specific first, and the first value obtained is injected. A command has one parent, so several `@Parent` members are alternative ways to take it, and only one is set.

The runtime's `save` command takes its parent two ways:

Listing 14. Two ways to take a parent (`SaveModelCommand.java`)

```
/**
 * A parent that produces the objects when asked, inside this command's span: preferred, and
 * the right shape for contents that are expensive to obtain.
 */
@Parent
private ExecutionSupplier<? extends Collection<? extends EObject>> contentsSupplier;

/**
 * A parent that is an {@link EObjectSupplier}. One parent, two ways to take it: only one of
 * these fields is injected.
 */
@Parent
private EObjectSupplier<? extends EObject> eObjectSupplier;
```

Ask with wildcards, `? extends` at every level that is only read from: each wildcard admits more parents. `Supplier<? extends CharSequence>` takes a supplier of strings and of string builders; `Supplier<CharSequence>` takes only suppliers declared exactly so.

6.3. What is expensive, ask for lazily

A parent that loads a model, calls a service or asks an AI model should do it only if a command needs it, and inside that command's span, so that the cost is attributed to the command. `ExecutionSupplier<T>` is a supplier that takes the executing command's span, logger and meter:

Listing 15. A parent that loads when asked, and a command that asks (`WiringTests.java`)

```
@Command(name = "catalog")
public static class CatalogCommand extends CommandGroup {

    int loads;

    @Adapter
    public ExecutionSupplier<List<EObject>> contents() { ①
        return (span, logger, meter) -> Telemetry.inSpan(
            Telemetry.tracer(Telemetry.current()),
            "Load the catalog",
            null,
            load -> {
                ++loads;
                return List.of(EcoreFactory.eINSTANCE.createEPackage());
            });
    }
```

```
}

@Command(name = "count")
public static class CountCommand extends CommandBase {

    @Parent
    ExecutionSupplier<? extends Collection<? extends EObject>> contents; ②

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) throws Exception {
        return contents.get(span, logger, meter).size(); ③
    }

}
```

① The offer: an `ExecutionSupplier` of a list of model objects, whose work is a span of its own.

② The need: an `ExecutionSupplier` of any collection of any model objects.

③ The work happens here, in the command's execution, with the command's span as the parent.

Listing 16. Loaded once, in the command's span

```
List<String> started = new ArrayList<>();
TracerProvider noop = TracerProvider.noop();
try (PublishingOpenTelemetry otel = new PublishingOpenTelemetry(
        OpenTelemetry.noop(),
        new PublishingTracerProvider(noop, Runnable::run, 64));
        Scope scope = Telemetry.makeCurrent(otel)) {
    otel.subscribe(new Flow.Subscriber<SpanEvent>() {

        @Override
        public void onSubscribe(Flow.Subscription subscription) {
            subscription.request(Long.MAX_VALUE);
        }

        @Override
        public void onNext(SpanEvent event) {
            if (event instanceof SpanEvent.Started s) {
                var parent = s.span().getParent();
                started.add(s.name() + " in " + (parent == null ? "-" : parent.getName()));
            }
        }

        @Override
        public void onError(Throwable throwable) {}

        @Override
        public void onComplete() {}

    });

    CatalogCommand catalog = new CatalogCommand();
```

```
    assertThat(commandLine(catalog, new CountCommand()).execute("count", "--help")).isZero();
    assertThat(catalog.loads).isZero(); ①

    assertThat(commandLine(catalog, new CountCommand()).execute("count")).isEqualTo(1);
    assertThat(catalog.loads).isEqualTo(1);
}
assertThat(started).containsExactly("count in -", "Load the catalog in count"); ②
```

① Asking for help runs nothing: nothing is loaded.

② `Load the catalog` is a child of `count`, the command that needed it.

6.4. What a command accepts as sub-commands

`@SubCommand` on a field, or on a method taking one parameter, says what a command accepts as sub-commands. A member typed as an array, a `List`, a `Collection`, an `Iterable` or a `Set` takes all the sub-commands adaptable to its element type; any other member takes the first.

Listing 17. Sub-commands by type (`WiringTests.java`)

```
public interface Exporter {

    String format();

}

@Command(name = "json")
public static class JsonCommand extends CommandBase implements Exporter {

    @Override
    public String format() {
        return "json";
    }

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) {
        return 0;
    }

}

@Command(name = "export")
public static class ExportCommand extends CommandBase {

    @SubCommand
    List<Exporter> exporters; ①

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) {
        spec.commandLine().getOut().print(exporters.stream().map(Exporter::format).toList());
```

```
        return 0;
    }

}
```

① Every sub-command that is an `Exporter`.

Listing 18. Only the exporters

```
assertThat(CommandWiring.acceptsSubCommand(ExportCommand.class, JsonCommand.class)).isTrue();
assertThat(CommandWiring.acceptsSubCommand(ExportCommand.class, CountCommand.class)).isFalse();

StringWriter out = new StringWriter();
CommandLine export = commandLine(new ExportCommand(), new JsonCommand(), new CountCommand());
export.setOut(new PrintWriter(out, true));
assertThat(export.execute()).isZero();
assertThat(out.toString()).isEqualTo("[json]"); ①
```

① `count` is a sub-command too, but not an exporter, so the list has `json` only.

When the tree is built, a sub-command goes under a command that accepts its class through a `@SubCommand` member, as well as under a parent its own `@Parent` members take. The two are the two directions of one relation: a child that knows what it needs, and a parent that knows what it accepts.

Sub-commands not on the command line have not had their options parsed when they are injected. They are there to be inspected, listed or delegated to, as the export command lists its formats.

6.5. Offers and hand-downs

What a parent offers with `@Adapter`, it offers to every command that might match: any command that needs that type goes under it. Sometimes a parent has something only for children written for it. A plain public method, not an `@Adapter`, is an offer only to children that take the parent by its class, `@Parent ExportCommand export`, and call the method.

Offer with `@Adapter` what any child may use; hand down with plain methods what only children written for this parent should use.

6.6. When a parent cannot provide

An adapter may decline at run time by returning null:

Listing 19. A parent that has nothing to supply (`WiringTests.java`)

```
@Command(name = "empty")
public static class EmptyCommand extends CommandGroup {

    @Adapter
    public ExecutionSupplier<List<EObject>> contents() {
        return null; // Declines
    }

}
```

Listing 20. Execution fails, help does not

```
CommandLine empty = commandLine(new EmptyCommand(), new CountCommand());
assertThat(empty.execute("count")).isEqualTo(CommandLine.ExitCode.SOFTWARE); ①
assertThat(empty.execute("count", "--help")).isEqualTo(CommandLine.ExitCode.OK); ②
```

① The command cannot be wired, and picocli reports the failure with the exit code for a software error. The message names the command, the types it needed, and the parent's class.

② Help skips wiring, so asking for help never fails on a parent that cannot provide.

6.7. When wiring happens

`WiringExecutionStrategy` injects the `@Parent` and `@SubCommand` members of the commands on the command line, and the `@Parent` members of their mix-ins, and then executes as its delegate does, picocli's `RunLast` by default.

- **After parsing**: a parent's options are parsed by then, and its adapters see their values.
- **From the root to the executed command**: a parent's own members are in place before it is adapted for its child, so a value can flow down several levels.
- **Not for help or version requests.**

The factories set the strategy on the root commands they create. A command line assembled by hand needs `commandLine.setExecutionStrategy(new WiringExecutionStrategy())`, as the tests of this chapter do.

6.8. Rules

- Members are found once per class, on the class and its superclasses, and must be accessible: in a package opened to the runtime, as picocli needs anyway. An inaccessible member fails with a message that names the package to open.
- A `@Parent` or `@SubCommand` member is a field or an instance method with one parameter, never static.
- `@Parent` is unrelated to picocli's `@ParentCommand`, which keeps working as picocli defines it.



> "The most specific first" orders members by the specificity of their erased types, so two members of the same raw type, `Supplier<String>` and `Supplier<CharSequence>`, tie and are ordered by name. The same tie is described in the Zephyr Book *Matching and Wiring*; the fix belongs in `Types` in the `common` module.

Chapter 7. Mix-ins

A picocli mix-in is a reusable set of options: a class with `@Option` fields, mixed into a command with `@Mixin` or `addMixin`. In this module, mix-ins can also be contributed: a factory adds a mix-in to the commands it matches, the way a command factory places a command.

7.1. A contributed mix-in

Listing 21. A mix-in for commands that supply text, and its factory (`ContributionTests.java`)

```
public static class ShoutMixIn {

    @Option(names = "--shout", description = "Upper case")
    boolean shout;

    @Parent
    Supplier<? extends CharSequence> text; ①

    String apply(String value) {
        return shout ? value.toUpperCase() : value;
    }

}

public static class ShoutMixInFactory extends MixInCapabilityFactory<ShoutMixIn> {

    @Override
    protected String getName() {
        return "shout"; ②
    }

    @Override
    protected Class<ShoutMixIn> getMixInType() {
        return ShoutMixIn.class;
    }

    @Override
    protected Publisher<ShoutMixIn> doCreateMixIn(
            List<CommandLine> commandPath,
            Loader loader,
            Span span,
            Logger logger,
            Meter meter) {
        return Flows.of(new ShoutMixIn());
    }

}
```

① For a mix-in, the parent is the command it is mixed into: this one goes into any command that supplies some kind of text, and is injected with the command's supplier.

② The mix-in's name, by which picocli knows it.

Listing 22. Mixed into the text group only

```
CommandLine root = assemble(
    new RootCommandFactory(),
    new TextCommandFactory(),
    new ModelCommandFactory(),
    new PrintCommandFactory(),
    new ShoutMixInFactory());
CommandLine text = root.getSubcommands().get("text");
CommandLine model = root.getSubcommands().get("model");
assertThat(text.getMixins()).containsKey("shout"); ①
assertThat(model.getMixins()).doesNotContainKey("shout");

assertThat(PipelineTests.run(root, "text", "--shout", "print")).isEqualTo("Hello");
ShoutMixIn shout = (ShoutMixIn) text.getMixins().get("shout");
assertThat(shout.text.get()).isEqualTo("Hello"); ②
assertThat(shout.apply("quiet")).isEqualTo("QUIET");
```

① The text group got the mix-in, and its `--shout` option; the model group did not.

② The mix-in's parent was injected with the group's supplier when the command line ran.

7.2. Mix-ins by type

The other direction is the command choosing its mix-ins. `@MixIns` on a field, or on a method taking one parameter, says what mix-ins a command accepts, by the member's generic type, as `@SubCommand` does for sub-commands; and the accepted mix-ins are injected into the member.

Listing 23. Two formatters, of text and of model elements (`LifecycleTests.java`)

```
public interface Formatter<T> {

    String format(T value);

}

public static class UpperFormatter implements Formatter<String> { ①

    @Option(names = "--upper")
    boolean upper;

    @Override
    public String format(String value) {
```

```
        return upper ? value.toUpperCase() : value;
    }

}

public static class NameFormatter implements Formatter<EObject> { ②

    @Override
    public String format(EObject value) {
        return value.eClass().getName();
    }

}
```

① A `Formatter<String>`, with an option of its own.

② A `Formatter<EObject>`: the same interface, another type argument.

Listing 24. A command that accepts formatters of text

```
@Command(name = "echo")
public static class EchoCommand extends CommandBase {

    @Parent
    Supplier<? extends CharSequence> text;

    @MixIns
    List<Formatter<? super String>> formatters; ①

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) {
        String value = text.get().toString();
        for (Formatter<? super String> formatter: formatters) {
            value = formatter.format(value);
        }
        spec.commandLine().getOut().println(value);
        return 0;
    }

}
```

① Every contributed mix-in that can format a `String`: a `Formatter<String>`, or a `Formatter<Object>` or `Formatter<CharSequence>`, which can format strings too. `? super` is the wildcard of a consumer, as `? extends` is of a producer.

Listing 25. Only the text formatter

```
CommandLine root = ContributionTests.assemble(
    new RootCommandFactory(),
    new TextCommandFactory(),
    new EchoCommandFactory(),
```

```
    new UpperFormatterFactory(),
    new NameFormatterFactory());
CommandLine echo = root.getSubcommands().get("text").getSubcommands().get("echo");
assertThat(echo.getMixins()).containsKey("upper").doesNotContainKey("name"); ①
assertThat(PipelineTests.run(root, "text", "echo", "--upper")).isEqualTo("HELLO"); ②
```

① The text formatter was mixed in, and the model formatter, the same class at run time, was not.

② Its option is the command's option, and the command applies it.

A member typed as an array, a `List`, a `Collection`, an `Iterable` or a `Set` takes all the accepted mix-ins; any other member takes the first.

7.3. Matching mix-ins

A mix-in factory adds its mix-in to a command when:

- the mix-in takes the command through a `@Parent` member, matched by type;
- the command accepts the mix-in through a `@MixIns` member, matched by type; or
- the command's class, or a superclass, is annotated with `@MixIns` listing types, and the mix-in is adaptable to one of them. Classes cannot carry type arguments, so this form matches raw types only, and nothing is injected: the command finds its mix-ins through picocli, `spec.mixins()`.

7.4. Mix-ins with fields

Mix-ins a command always has need no factory: a picocli `@Mixin` field is enough, and the runtime's `ResourceSetMixIn` and `PropertiesMixIn` are used that way (Chapter 9). Contribute a mix-in when the command should not have to know it: an option that a module adds to every command that supplies a model, for example.

7.5. Mix-in or command

The previous generation of Nasdanika relied on mix-ins for separation of concerns, and it led to option explosion and name collisions: every concern added options to every command. A sub-command is often the better unit: it has its own options, it appears only where it applies, and it composes into pipelines. I usually write the base of a concern as a class without `@Command`, so it

can be mixed in, and a command that extends it, so it can be a step of a pipeline; which one to contribute is a judgement call.

Chapter 8. Overriding

Two modules may contribute a command with the same name under the same parent: a better `print`, a `save` that knows a corporate repository. Picocli refuses two sub-commands with one name, so the runtime keeps one when it can tell which overrides the other, the way a subclass overrides a method.

8.1. By subclassing

Listing 26. A command that overrides another by extending it (`ContributionTests.java`)

```
@Command(name = "print", description = "Prints its parent's text, framed")
public static class FramedPrintCommand extends PrintCommand { ①

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) {
        spec.commandLine().getOut().println("[" + text.get() + "]");
        return 0;
    }

}
```

① The same name, `print`, and a subclass of the command it replaces.

8.2. By declaration

Listing 27. A command that declares what it overrides

```
@Command(name = "print", description = "Prints nothing")
@Overrides(PrintCommand.class) ①
public static class SilentPrintCommand extends CommandBase {

    @Parent
    Supplier<? extends CharSequence> text;

    @Override
    protected Integer execute(Span span, Logger logger, Meter meter) {
        return 0;
    }

}
```

① Overrides any `PrintCommand`, without extending it.

Listing 28. The overriding command is kept

```
CommandLine framed = assemble(
    new RootCommandFactory(),
    new TextCommandFactory(),
    new PrintCommandFactory(),
    new FramedPrintCommandFactory());
assertThat(print(framed)).isInstanceOf(FramedPrintCommand.class); ①
assertThat(PipelineTests.run(framed, "text", "print")).isEqualTo("[Hello]");

CommandLine silent = assemble(
    new RootCommandFactory(),
    new TextCommandFactory(),
    new PrintCommandFactory(),
    new SilentPrintCommandFactory());
assertThat(print(silent)).isInstanceOf(SilentPrintCommand.class); ②
```

① With both factories contributing, `print` is the framed one.

② And with the silent one, the silent one.

8.3. The rules

Of commands with the same name under the same parent, a command overrides another when:

- its class is annotated with `@Overrides`, and the other is an instance of one of the listed types;
- it implements `Overrider`, and `overrides(other)` returns true;
- its class is a subclass of the other's class.

Commands that do not override each other are all kept, and picocli reports the duplicate name. The same rules apply to mix-ins with the same name.

Overriding is what makes a command line polymorphic: a distribution for one audience adds a module whose commands replace the defaults, and nothing else changes.



`SubCommandSelector`, for a command to choose among sub-commands with the same name, and `SubCommandPredicate`, for a command to accept or refuse a sub-command, are public interfaces of the module that nothing calls. Either wire them into `SubCommandCapabilityFactory`, or remove them.

Chapter 9. The runtime's commands

9.1. Commands

Command	What it does
`nsdk`	The root command, with version information from the module descriptors and Git properties, when a module has them
`launcher`	Generates a Java command line, or its options, from directories of modules and jars (Section 9.5)
`save`	Saves its parent's model objects to a file, in any format the runtime writes, chosen by the extension
`ecore`	A group that narrows its parent's model objects to Ecore model elements, and supplies them to its own sub-commands
`shell`	An interactive shell over the root command, with `exit`; added by the application and the launcher

`save` goes under any command that supplies model objects, through an `ExecutionSupplier` of a collection of them or an `EObjectSupplier` (Chapter 6), and `ecore` under any `EObjectSupplier` that is not itself a filter. So under a command that supplies a model, `save` saves all of it and `ecore save` saves its Ecore elements, with no command knowing the others.

9.2. The shell

`shell` reads lines with JLine, splits them into arguments with quotes, and executes each line against the root command, in the same JVM, so that models loaded once stay loaded and the start-up cost is paid once:

Listing 29. How a shell line is split (`WiringTests.java`)

```
assertThat(ShellCommand.parseLine("text --value 'Hi there' print", null, null))
    .containsExactly("text", "--value", "Hi there", "print");
```

9.3. Mix-ins

Mix-in	Options
`ResourceSetMixIn`	`--extension-resource-factory`, `--protocol-resource-factory` and `--content-type-resource-factory`, which map an extension, a protocol or a content type to a resource factory class; `createResourceSet()` returns a resource set configured from the module path and the options, and `createResourceSet(configurator)` lets the caller configure it last
`PropertiesMixIn`	`-p name=value` for one property, `-P` for a properties resource, in YAML, JSON or properties format by extension or content type, loaded in order, later ones replacing earlier ones



`TelemetryMixIn` is an empty placeholder with TODOs for saving and serving telemetry; describe it when it has options.

9.4. Telemetry

Every executed command is a span named after it, a child of the span current when it ran: the launcher's root span in a distribution. The work a command does, resource loads and saves, capability resolution, content transfers, a parent's work done through an `ExecutionSupplier`, is in child spans, and log records it emits are in its span. Building the tree is traced too: each command line created is a log record, with its parent path and the running total of commands.

9.5. Distributions and the launcher

A distribution is a directory of jars, in the Maven repository layout, and a way to start the JVM with the right module path. The runtime's `cli-launcher` module builds the distribution of `nsdk`, and the same machinery builds a distribution of a command line of your own:

- the Maven dependency plugin copies the runtime dependencies into `target/dist/lib`;
- the `launcher` command scans directories of jars, resolves the modules, and writes the Java command line, or just its options, with the named modules on the module path and the rest on the class path;
- wrapper scripts start the JVM with those options from any directory.

Option	Meaning
`[DIRECTORIES]`	Directories to scan for modules, `lib` by default
`-b`, `--base`	The base directory of the repositories and of the output
`-o`, `--output`	The file to write the command line to; standard output if absent
`-t`, `--options`	Write only the Java options, not the command
`-f`, `--options-file`	A file to write the options to
`-r`, `--root-modules`	Root modules, with `. for the modules in a package and .*` for those below it
`--add-modules`	Modules for the Java `--add-modules` option, computed if absent
`-M`, `--modules`	A file listing modules to add to the module path
`-C`	Modules to put on the class path
`-j`, `--java`	The Java command, `java` by default, with any extra options such as system properties
`-a`, `--args`	The arguments placeholder, `%*` by default
`-m`, `--module`, `-c`, `--class`	The application's module and main class, the `cli` module's `Application` by default
`-p`, `--path-separator`, `-s`, `--absolute`, `-P`, `--prefix`	Path separator, absolute paths, and a prefix for module and class path entries



The long name of `-C` is spelled `--claspath-modules`, with one "s". Correct it to `--classpath-modules`, keeping the misspelling as an alias for a release.

Part III: Looking Ahead

Pipelines that are being built, and those that are planned.

Chapter 10. Pipelines

10.1. What the previous generation built

In 2024, Nasdanika's command line used declarative assembly, by classes then, to build pipelines of commands from different modules. The one in daily use generated sites from Draw.io diagrams: `drawio` loaded a document and was a `Document.Supplier`; `html-app` went under any `Document.Supplier` and produced an application model, a `LabelSupplier`; `site` went under any `LabelSupplier` and generated the site; and `save` went under all of them, as an `EObjectSupplier` consumer, to save any intermediate model. None of the commands knew the others. The capability framework was used in two dimensions: horizontally, to link commands into pipelines, and vertically, to give each command what it needed, such as a resource set with the packages and resource factories of the modules on the path, so that a model command did not need to know that a spreadsheet was a model.

10.2. Being rebuilt on types

The runtime rebuilds those pipelines on generic types, without the marker interfaces. **SKETCH**: the `save` and `ecore` commands are in place, and so are the modules the pipelines work on, the runtime's Draw.io API, diagram model and telemetry model; the commands that load Draw.io documents and diagrams, convert them and generate sites from them are not migrated yet.

10.3. Planned

PLANNED:

- **Scripts and a Groovy shell** under any command that supplies objects, to interrogate a model of millions of objects, Git projects, commits and files, or classes, methods and fields, interactively and incrementally, in the manner of Naked Objects, instead of writing a command for each question.
- **Chats** with a project, a set of diagrams, a model or a generated site: a chat command supplying a conversation, and command line and REST sub-commands going under any command that supplies one.
- **HTTP serving** separated from routing: commands that build routes, a web interface or a

REST API, and an `http` command going under any of them, on the runtime's HTTP handlers and with a harness server in tests.

- **Command lines as models**: a Groovy DSL for long pipelines, and forms generated from command definitions, sub-commands as wizards and mix-ins as sections.
- **Telemetry options** for every command, saving a run's telemetry to a file or serving it live, from `TelemetryMixIn`.

Each is a command or two in a module of its own, placed by what it consumes, and each makes the others more useful: a chat about a model generated from a diagram, saved after it was filtered.

Appendix A: Reference

A.1. Writing commands

Type	Purpose
`CommandBase`	Base of commands: runs `execute(span, logger, meter)` in a span named after the command, and closes its `Closeable` sub-commands and mix-ins; `CommandBase.close(commandLine)` closes a whole command line
`CommandGroup`	A command that does nothing itself and fails with "Missing required subcommand" when run alone
`@Parent`	On a field or a one-parameter method: what the command, or the mix-in, needs from its parent, by generic type
`@SubCommand`	On a field or a one-parameter method: what the command accepts as sub-commands, by generic type; collections, sets and arrays take all, other types the first
`ExecutionSupplier<T>`	A value produced in the executing command's span: `T get(span, logger, meter)`
`@MixIns`	On a field or a one-parameter method: what mix-ins the command accepts, by generic type, injected like sub-commands; on a command class: raw types of mix-ins to add to it
`@Overrides`, `Overrider`	Declaring or deciding which commands and mix-ins of the same name this one overrides

A.2. Contributing

Type	Purpose
`SubCommandCapabilityFactory<T>`	Base of command factories: `getCommandType()`, `doCreateCommand(⋯)`, and `match(⋯)` or `createCommand(⋯)` to decide in code
`MixInCapabilityFactory<T>`	Base of mix-in factories: `getName()`, `getMixInType()`, `doCreateMixIn(⋯)`, and `match(⋯)`
`SubCommandRequirement`	The requirement for commands: a parent path, empty for root commands, and a command counter

Type	Purpose
`MixInRequirement`, `MixInRecord`	The requirement for mix-ins of a command, and a named mix-in
`CommandWiring`	Matching, `acceptsParent` and `acceptsSubCommand`, and injection of `@Parent` and `@SubCommand` members
`WiringExecutionStrategy`	Injects after parsing, from the root to the executed command, then runs picocli's `RunLast`

A.3. Running

Type	Purpose
`Application`	A main class: builds the tree from a module layer, adds the shell, and executes the arguments
`RootCommand`, `RootCommandFactory`	The root, `nsdk`, created for the empty parent path
`ShellCommand`, `ExitCommand`	The interactive shell, and its `exit`
`LauncherCommand`, `AbstractLauncherCommand`, `AbstractModuleResolverCommand`	Java command lines and options files from directories of modules
`ModuleVersionProvider`	Version help from the module descriptor and `META-INF/git.properties`
`SaveModelCommand`, `EcoreCommand`	The runtime's `save` and `ecore`, with their factories
`ResourceSetMixIn`, `PropertiesMixIn`	Options for resource factories, and for properties

A.4. System properties

Property	Default	Meaning
`org.nasdanika.sdk.runtime.cli.maxCommandPath`	15	The longest parent path a command is created for
`org.nasdanika.sdk.runtime.cli.maxCommands`	2000	The most commands in one tree

Appendix B: Building this book

This book is AsciiDoc in `cli/src/docs/asciidoc`, built with the module after its tests have passed:

```
mvn -pl cli verify                              # the module, its tests and this book
mvn -pl cli verify -Dzephyr.editor-notes=false  # without editor's notes
mvn -pl cli verify -Dasciidoctor.skip=true      # without the book
```

The PDF, the single-page HTML and the EPUB are written to `target/book`; the PDF and the HTML are attached to the module's artifact with the classifier `book`, and the test sources with the classifier `test-sources`. The look comes from the runtime's `doc/zephyr` directory, shared by all the Nasdanika Zephyr Books.

Listings are tagged regions of the tests in `src/test/java/org/nasdanika/sdk/runtime/cli`, `PipelineTests`, `WiringTests`, `ContributionTests` and `LifecycleTests`, and of the module's `module-info.java` and `SaveModelCommand.java`. The tests are in the module's own package because it is open: picocli and the wiring reach private members of commands, and a test package would have to be opened to them as well. A listing line holds at most 96 characters, and a warning fails the build.

Appendix C: Glossary

Adapter

A field or method annotated `@Adapter`, whose generic type is what its object offers; for a command, what it offers its sub-commands.

Capability factory

An object contributed as a service that creates capabilities for requirements it handles; here, commands and mix-ins for places in the tree.

Command line

Picocli's `CommandLine`: a command, its options, its sub-commands and its mix-ins.

Command tree

The commands of a command line from the root down, assembled from what the module path contributes.

Execution supplier

A supplier called in the executing command's span, for what is expensive to produce.

Group

A command that does nothing itself, and organizes and supplies its sub-commands.

Internal DSL

A language for a domain made of a host language's own constructs: picocli's annotations for command lines in Java.

Mix-in

A reusable set of options, mixed into commands; here also contributed by factories.

Overriding

Keeping one of several commands or mix-ins with the same name, the one that overrides the others.

Parent path

The commands from the root to the place where a command is being created.

Pipeline

A sequence of commands, each consuming what the previous one supplies, written as one command line.

Sensory bias

In evolutionary biology, a preference evolved for one purpose and exploited for another; here, Java developers' fluency with annotations and generic types, used to write command lines.

Wiring

Placing commands by their declared needs, and injecting what their parents supply.